TOOLKIT · BACKEND ENGINEERING HANDBOOK

POSTMAN COMPLETE HANDBOOK

API Testing, Collections, Environments, Scripting, Chaining, Newman & CI/CD Automation

Phase	Tool ID	Curriculum Scope	Deliverable Type
I — Developer Foundations	**Tool 04 of 13 (Postman)**	**Complete API Engineering**	**Publication Handbook**
Interview Questions	**Practical Lab**	**Engineering Framework**	**Target Audience**
15 Rigorous Questions	**Hands-On Capstone Lab**	**3-Layer Architecture**	**Backend Engineers & SDE-1**

Executive Notice: This handbook is the authoritative API testing and automation volume from the *ToolKit* curriculum. It covers the HTTP protocol lifecycle, status code semantics, request anatomy, Postman collections and environments, reusable variable scoping, dynamic request chaining, automated JavaScript assertions via pm.test, security & IDOR testing, Newman CI/CD test automation, production 500 error incident forensics, 15 core interview questions, and a hands-on capstone.

Table of Contents

LAYER 1 — FUNDAMENTALS & MENTAL MODELS

1. What Is Postman? (The API Request-Response Cycle & Workbench Architecture) Section 1
2. Understand an HTTP Request (Methods, Headers, URL Path & Query Parameters) Section 2
3. Understand an HTTP Response (Status Codes 2xx, 4xx, 5xx & Headers) Section 3
4. Postman Terminology (Requests, Collections, Environments, Variables, Runners) Section 4
5. Postman versus Other Testing Tools (JUnit 5, Spring Boot Test, curl, Load Tools) Section 5

LAYER 2 — PRACTICAL ENGINEERING & WORKFLOWS

6. Send Your First Request (Endpoint Triage & Spring Boot Actuator) Section 6
7. Build an Orders API Collection (Hierarchical Resource Organization) Section 7
8. Use Variables and Environments (Local, Staging & Dynamic Scopes) Section 8
9. Send a POST Request with JSON Payloads (Headers & Bearer Tokens) Section 9
10. Write Automated Assertions (pm.test, pm.expect & Schema Validation) Section 10
11. Chain Requests Together (State Extraction & Dynamic ID Passing) Section 11
12. Test Authentication and Authorization (401 vs. 403 & IDOR Vulnerabilities) Section 12
13. Test Validation and Edge Cases (RFC 7807 Problem Details & Type Errors) Section 13
14. Test Pagination and Filtering (Page Numbers, Sizing & Boundaries) Section 14
15. Run a Collection as a Test Suite (Collection Runner & State Isolation) Section 15
16. Export and Share Safely (Sanitization, Secrets & Git Hygiene) Section 16

LAYER 3 — PRODUCTION ENGINEERING & INTERVIEW MASTERY

17. Build a Production-Grade API Test Strategy (Four-Quadrant Architecture) Section 17
18. CI Integration with Newman (Headless Automation & GitHub Actions) Section 18
19. Production Incident Scenario: API Returns 500 (Forensic Investigation) Section 19
20. Performance & Reliability Limitations (Functional Testing vs. Load Testing) Section 20
21. Common Testing Mistakes (Anti-Patterns to Avoid) Section 21
22. Interview Questions — 15 Essential Questions with Detailed Answers Section 22
23. Hands-on Capstone Lab & Final Mastery Checklist Section 23

Postman — Backend Engineering Tools & Infrastructure Handbook

Phase: I — Developer Foundations

Chunk: 1 of 1

Source file: `content/postman/01-postman-api-testing-workflows-production.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain how Postman fits into backend API development and debugging.
- Send HTTP requests and inspect responses.
- Work with collections, environments, variables and authentication.
- Write automated assertions and chain requests together.
- Test error handling, authorization and edge cases.
- Run API tests in CI and troubleshoot production-like failures.

Layer 1 — Fundamentals & Mental Models

1. What is Postman?

Imagine you're developing an orders API with Spring Boot.

Your backend exposes endpoints such as:

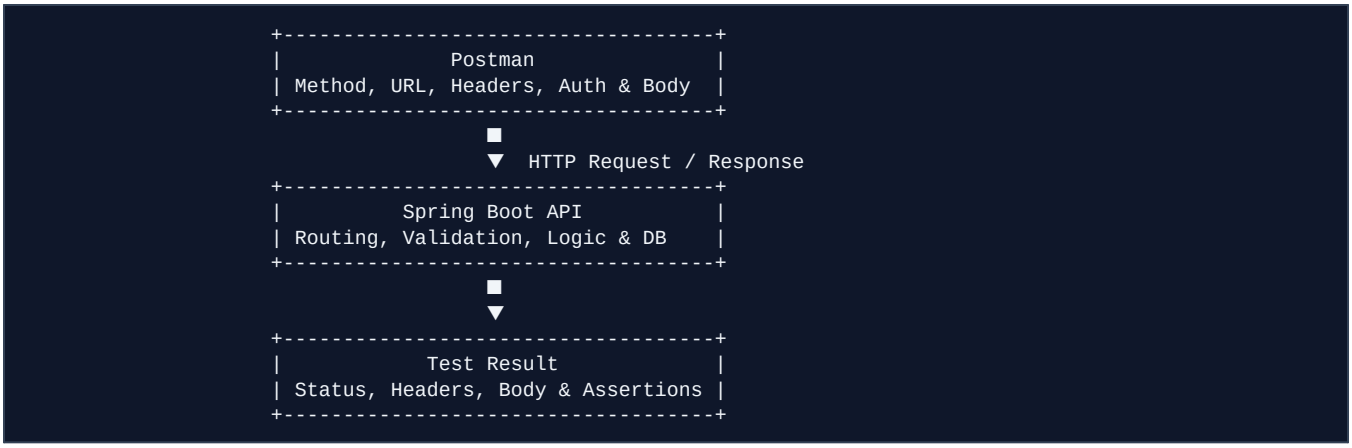
```
POST   /api/orders
GET    /api/orders/123
GET    /api/orders
DELETE /api/orders/123
```

Before building a frontend or integrating another service, you need to verify that these endpoints behave correctly.

Postman is an API development and testing platform that lets you construct requests, send them to a server, inspect responses and automate API checks.

Think of it as a workbench for communicating directly with your backend.

The API Request-Response Cycle



Postman does not replace unit tests or integration tests. It complements them by making it easy to exercise an API through its external HTTP interface.

---

2. Understand an HTTP request

Consider this request:

```
POST /api/orders HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Authorization: Bearer YOUR_ACCESS_TOKEN

{
  "productId": "P-101",
  "quantity": 2
}
```

A request has several important components:

Component	Purpose	Typical Engineering Use
Method	Specifies the intended operation	`GET`, `POST`, `PUT`, `PATCH`, `DELETE`
URL	Identifies the target server and resource	Scheme (`http/https`), host, port, path
Headers	Carry protocol and application metadata	`Content-Type`, `Accept`, `Authorization`
Query parameters	Filter, search, paginate or modify request selection	`?page=0&size=20&status=PENDING`
Path parameters	Identify a specific resource or nested entity	`/api/orders/{orderId}/items/{itemId}`
Body	Carries input payload when the operation requires it	JSON, form-data, XML payload
Authentication	Supplies credentials or tokens for access	Bearer JWT tokens, API keys, Basic auth

Common HTTP methods

- `GET`: retrieve a resource or collection. Must be safe and idempotent.
- `POST`: submit data or request an operation, commonly creating a resource. Non-idempotent.
- `PUT`: replace a resource representation entirely. Idempotent.

- **`PATCH`**: partially update a resource. Not necessarily idempotent.
- **`DELETE`**: request removal of a resource. Idempotent.

The precise behavior depends on the API contract. HTTP method semantics matter when designing tests for safety and idempotency.

---

3. Understand an HTTP response

A response contains a status code, headers and possibly a body:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/orders/123

{
  "id": "123",
  "status": "CREATED",
  "quantity": 2
}
```

Important Status Codes

Code	Meaning	Typical Backend API Use
`200`	OK	Successful request returning representation or message
`201`	Created	Resource successfully created (often with `Location` header)
`204`	No Content	Successful operation with no entity body (e.g. `DELETE`)
`400`	Bad Request	Syntactically or structurally invalid input from client
`401`	Unauthorized	Missing, expired, or invalid authentication credentials
`403`	Forbidden	Authenticated caller lacks permissions (insufficient role)
`404`	Not Found	Target endpoint route or database resource ID does not exist
`409`	Conflict	State or uniqueness violation (e.g. duplicate email/key)
`422`	Unprocessable Content	Syntactically valid JSON failing business validation rules
`429`	Too Many Requests	Rate limit quota exceeded
`500`	Internal Server Error	Unhandled server exception or unexpected failure
`503`	Service Unavailable	Downstream dependency outage or overloaded backend service

Your tests should assert the exact status code required by the API contract, rather than assuming every successful request returns `200`.

---

4. Postman terminology

- **Request:** one configured API call (method, URL, headers, body, scripts).
- **Collection:** a structured group of related requests representing an API suite or workflow.
- **Environment:** a set of key-value variables tied to a target infrastructure (e.g. `local`, `staging`, `production`).
- **Variable:** a reusable placeholder such as `{{baseUrl}}` or `{{orderId}}` scoped to global, environment, collection, or local levels.
- **Pre-request script:** JavaScript executed immediately before the request is transmitted.
- **Post-response script:** JavaScript executed immediately after the response arrives, used for assertions and extracting state.
- **Runner:** an execution engine that runs entire collections or folders sequentially with data files.
- **Mock server:** a simulated API endpoint returning configured example responses without hitting real backend servers.
- **Monitor:** scheduled collection runs executing tests periodically from Postman cloud workers.

A collection can become a reusable API test suite instead of a collection of manually executed requests.

---

5. Postman versus other testing tools

Tool	Primary Role & Sweet Spot	Comparison with Postman
Postman	Interactive API exploration and automated request-level contract checks	Black-box testing over external HTTP; visual and scriptable
JUnit 5	Java testing framework for unit and component-level assertions	White-box Java testing running within the JVM process
Spring Boot Test	Integration testing with MockMvc, WebTestClient, or Testcontainers	In-memory servlet testing with direct access to Spring application context
`curl`	Command-line HTTP utility for rapid server-side connectivity checks	Lightweight CLI tool; lacks native GUI, collection runners, and test suites
Load-testing tools (JMeter, k6, Gatling)	Simulating thousands of concurrent virtual users for performance profiling	High-concurrency throughput testing vs Postman's functional verification

For a backend engineer, these tools complement one another. Postman is especially useful for verifying the behavior exposed to API consumers.

Layer 2 — Practical Engineering & Workflows

6. Send your first request

Suppose your Spring Boot application is running on port `8080`.

1. Open Postman and create a new HTTP request.
2. Select the `GET` method.

3. Enter `http://localhost:8080/actuator/health` (if Spring Boot Actuator is configured).
4. Click **Send**.
5. Inspect the status code (`200 OK`), response body (`{"status":"UP"}`), headers, and round-trip response latency.

If your application doesn't expose Actuator, use an existing endpoint or configure a dedicated health endpoint in your application controller.

If the request fails:

- Check whether the server process is listening (`ss -lntp | grep ':8080'`).
- Verify the port and URL route mappings in Spring logs.
- A network error in Postman does not automatically mean application code is defective; check host connectivity first.

---

7. Build an Orders API collection

Create a collection named `Orders API`.

Organize requests into logical folders mirroring REST resource paths:

```
Orders API
■■■ Health
■   ■■■ Check API Health
■■■ Orders
■   ■■■ Create Order (POST)
■   ■■■ Get Order by ID (GET)
■   ■■■ List Orders with Pagination (GET)
■   ■■■ Delete Order (DELETE)
■■■ Authorization
    ■■■ Request without Token (401)
    ■■■ Request with Insufficient Permissions (403)
```

Folder-based organization enables running targeted test suites (e.g. running only the `Authorization` folder) and configuring folder-level authorization inheritance.

---

8. Use variables and environments

Avoid hardcoding hostnames, ports, and tokens directly inside individual requests.

Create an environment named `Local Development` with:

Variable	Initial Value	Current Value	Scope
`baseUrl`	`http://localhost:8080`	`http://localhost:8080`	Environment
`orderId`	*(empty)*	`order-101` (set dynamically)	Collection
`accessToken`	*(empty)*	`eyJhbGci...`	Environment (Secret)

Then write your request URL using double curly braces:

```
{{baseUrl}}/api/orders
```

Switching environments from `Local Development` to `Staging` automatically redirects all requests in the collection to `https://staging-api.example.com` without editing single URLs.

Security Rule: Never store production credentials, cloud secrets, or permanent JWT tokens in exported collection JSON files committed to Git. Use environment-level secrets or external secret management tools.

---

9. Send a POST request with JSON

Configure the request:

- **Method:** `POST`
- **URL:** `{{baseUrl}}/api/orders`
- **Headers:** `Content-Type: application/json`
- **Body:** raw JSON

Request payload:

```
{
  "productId": "P-101",
  "quantity": 2
}
```

If the endpoint requires security, select **Bearer Token** in the Authorization tab and supply `{{accessToken}}`.

Example successful response (`201 Created`):

```
{
  "id": "order-123",
  "status": "CREATED",
  "productId": "P-101",
  "quantity": 2,
  "createdAt": "2026-10-23T08:30:00Z"
}
```

---

10. Write automated assertions

Sending a request only proves that you sent it. Automated assertions verify whether the response strictly satisfies the API contract.

In the request's **Scripts** → **Post-response** tab, write JavaScript tests using Postman's `pm.test` API:

```
pm.test("Status code is 201 Created", function () {
    pm.response.to.have.status(201);
});

pm.test("Response contains Content-Type application/json header", function () {
    pm.response.to.have.header("Content-Type");
    pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});

pm.test("Response body is valid JSON", function () {
    pm.response.to.be.json;
});

pm.test("Order payload has expected fields and types", function () {
    const body = pm.response.json();

    pm.expect(body.id).to.be.a("string").and.not.empty;
    pm.expect(body.status).to.eql("CREATED");
    pm.expect(body.productId).to.eql("P-101");
    pm.expect(body.quantity).to.eql(2);
});

pm.test("Response time is within acceptable SLA (under 500ms)", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
});
```

---

11. Chain requests together

In backend workflows, subsequent API operations depend on identifiers generated by previous requests:

```
1. Create Order (POST)  ■■>  Extract 'id'  ■■>  Store in {{orderId}}
                                                      ■
                                                      ▼
2. Get Order (GET)      <■■  Query /api/orders/{{orderId}}
```

In Step 1: Create Order (Post-response script)

```
pm.test("Order created successfully", function () {
    pm.response.to.have.status(201);
    const body = pm.response.json();
    pm.expect(body.id).to.be.a("string");

    // Persist to collection variable for downstream requests
    pm.collectionVariables.set("orderId", body.id);
});
```

In Step 2: Get Order by ID

Configure request URL as:

```
{{baseUrl}}/api/orders/{{orderId}}
```

In Step 2's Post-response script:

```
pm.test("Retrieved order matches previously created order ID", function () {
    pm.response.to.have.status(200);
    const body = pm.response.json();
    const expectedId = pm.collectionVariables.get("orderId");

    pm.expect(body.id).to.eql(expectedId);
    pm.expect(body.status).to.eql("CREATED");
});
```

Diagnostic Rule: If Step 1 fails, Step 2 will inevitably fail due to an undefined variable. Always investigate the *first* failed assertion in a chained workflow.

---

12. Test authentication and authorization

Security testing must verify authentication and role-based permissions:

Scenario	Request Configuration	Expected HTTP Status
Missing Token	Omit `Authorization` header completely	`401 Unauthorized`
Invalid Token	Provide malformed or fake JWT (`Bearer abc.123`)	`401 Unauthorized`
Expired Token	Provide timestamp-expired JWT token	`401 Unauthorized`
Insufficient Role	Provide valid user token calling admin endpoint	`403 Forbidden`
Valid Role	Provide authorized token	`200 OK` or `201 Created`

Postman Test Script for 401 Unauthorized

```
pm.test("Returns 401 Unauthorized when credentials are absent", function () {
    pm.response.to.have.status(401);
});
```

Insecure Direct Object References (IDOR) Testing

Verify object-level boundaries: User A must receive `403 Forbidden` or `404 Not Found` when attempting to access `/api/orders/{{orderIdOfUserB}}`.

---

13. Test validation and edge cases

Never test only happy paths. Rigorous backend engineering tests boundaries, malformed inputs, and type violations.

Test Scenarios for an Order Quantity Field:

- **Valid positive number:** `{"productId": "P-101", "quantity": 2}` → `201 Created`
- **Zero quantity:** `{"productId": "P-101", "quantity": 0}` → `400 Bad Request`
- **Negative quantity:** `{"productId": "P-101", "quantity": -5}` → `400 Bad Request`
- **String instead of number:** `{"productId": "P-101", "quantity": "two"}` → `400 Bad Request`

- **Extremely large integer:** `{"productId": "P-101", "quantity": 99999999999999}` → `400 Bad Request`
- **Malformed JSON:** `{productId: P-101,` → `400 Bad Request`
- **Missing required field:** `{"quantity": 2}` → `400 Bad Request`

Validate error payload schemas:

```
pm.test("Validation error conforms to RFC 7807 Problem Details", function () {
    pm.response.to.have.status(400);
    const body = pm.response.json();

    pm.expect(body).to.have.property("title");
    pm.expect(body).to.have.property("status", 400);
    pm.expect(body).to.have.property("errors");
    pm.expect(body.errors).to.be.an("array");
});
```

---

14. Test pagination and filtering

For listing endpoints with pagination parameters:

```
{{baseUrl}}/api/orders?page=0&size=10&status=CREATED
```

Assertions:

```
pm.test("Pagination and filtering parameters respected", function () {
    pm.response.to.have.status(200);
    const body = pm.response.json();

    pm.expect(body.content).to.be.an("array");
    pm.expect(body.content.length).to.be.at.most(10);
    pm.expect(body.pageNumber).to.eql(0);
    pm.expect(body.pageSize).to.eql(10);

    // Verify all returned orders match filtered status
    body.content.forEach(order => {
        pm.expect(order.status).to.eql("CREATED");
    });
});
```

---

15. Run a collection as a test suite

The **Collection Runner** executes all requests in a collection or folder sequentially:

1. Open Postman and click **Run Collection**.
2. Select target folder and target **Environment** (`Local Development`).
3. Set iterations and optional delay between requests (e.g. 100ms).
4. Click **Run Orders API**.
5. Inspect the test summary: total tests, passed, failed, and average latency.

Test Isolation Principles:

- Avoid assuming pre-existing database records.
- Let pre-request scripts or setup requests seed prerequisite data.
- Include a cleanup step (`DELETE /api/orders/{{orderId}}`) to prevent state pollution across test runs.

---

16. Export and share safely

Collections and environments can be exported as standard JSON files:

- `Orders-API.postman_collection.json`
- `local.postman_environment.json`

Sanitization Protocol Before Committing:

1. Clear **Current Value** fields for all tokens and secrets (Postman exports Initial Values by default; verify both).
2. Check pre-request and test scripts for hardcoded API keys.
3. Replace hardcoded IP addresses or internal domain names with `{{baseUrl}}`.
4. Store environment files containing sensitive credentials in secure secret vaults rather than public repositories.

Layer 3 — Production Engineering & Interview Mastery

17. Build a production-grade API test strategy

Organize your Postman test suites across four primary engineering quadrants:

Test Category	Scope & Objective	Engineering Focus
Happy-Path Tests	Confirm valid requests produce documented success responses and state transitions	`200`/`201` status codes, payload contract verification, ID generation
Security Tests	Verify authentication, authorization, ownership boundaries and data sanitization	`401`/`403` checks, invalid/expired tokens, IDOR isolation
Failure-Path Tests	Exercise boundary errors, missing resources, invalid types and rate limiting	`400 Bad Request`, `404 Not Found`, `409 Conflict`, `429 Too Many Requests`
Workflow Tests	Validate stateful sequences across multiple dependent endpoints	Create → Retrieve → Update → Delete with shared collection variables

---

18. CI integration with Newman

To run Postman collections in CI/CD build pipelines (GitHub Actions, GitLab CI, Jenkins), use **Newman**—Postman's official command-line collection runner.

Install Newman:

```
npm install -g newman
```

Execute Collection via CLI:

```
newman run Orders-API.postman_collection.json \
  -e environments/local.postman_environment.json \
  --reporters cli,junit \
  --reporter-junit-export target/newman-report.xml
```

GitHub Actions CI Workflow Example:

```
name: API Regression Tests

on: [push, pull_request]

jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'

      - name: Start Spring Boot Backend
        run: |
          ./mvnw clean package -DskipTests
          nohup java -jar target/*.jar > app.log 2>&1 &
          npx wait-on http://localhost:8080/actuator/health --timeout 60000

      - name: Install Newman
        run: npm install -g newman

      - name: Execute Postman Test Suite
        run: |
          newman run tests/Orders-API.postman_collection.json \
            -e tests/ci.postman_environment.json \
            --bail
```

The `--bail` flag instructs Newman to immediately stop execution on the first assertion failure, failing the CI build step.

---

19. Production incident scenario: API returns 500 Internal Server Error

Situation: A frontend client reports that submitting orders intermittently triggers `500 Internal Server Error`.

Postman Incident Diagnostic Workflow:

1. **Reconstruct exact request**: Match the exact HTTP method, headers (`Content-Type`, `X-Correlation-ID`), query params, and JSON body.
2. **Execute with Postman Console**: Open the Postman Console (`Alt+Ctrl+C` / `Cmd+Option+C`) to inspect raw outgoing wire bytes, SSL handshake details, and server response headers.
3. **Capture Correlation ID**: Check response headers for `X-Trace-Id` or `X-Request-Id`.

4. **Correlate with Server Logs**: Grep the backend server log for the captured correlation ID:

```
grep 'trace-abc-123' /var/log/orders-api/app.log -C 5
```

5. **Differentiate 500 Causes**:
   - `NullPointerException` due to an unexpected null field in the JSON payload.
   - Database connection timeout or deadlock.
   - Downstream payment gateway socket timeout.
6. **Caution with Non-Idempotent Operations**: Never mindlessly spam `Send` on failing `POST /api/orders` or `/api/payments` endpoints during an incident; doing so can create duplicate financial charges or orphaned inventory locks.

---

20. Performance and reliability limitations

Postman provides response round-trip time metrics, but it is **not** a load-testing engine.

Why Postman is Insufficient for Load Testing:

- Postman executes requests sequentially or across limited local threads.
- It cannot generate thousands of concurrent virtual users (VU) with ramp-up curves.
- It does not calculate percentile distributions ($p95$, $p99$) under distributed network strain.
- For true performance engineering, use dedicated tools: **k6**, **Apache JMeter**, or **Gatling**.

Distinguish between **Client Timeout** and **Server Failure**: If Postman reports `Error: ESOCKETTIMEDOUT` after 10 seconds, the backend server may still be executing the query and mutating database state in the background.

---

21. Common testing mistakes

1. **Asserting only status codes**: A response can return `200 OK` with an error payload (`{"error": "Failed"}`). Always assert payload schemas.
2. **Ignoring HTTP method semantics**: Using `POST` when updating existing resources instead of `PUT` or `PATCH`.
3. **Hardcoding dynamic IDs**: Hardcoding `123` instead of dynamically capturing generated IDs from previous responses.
4. **Environment variable pollution**: Overwriting production environment variables during local testing runs.
5. **Committing secrets**: Pushing API keys or bearer tokens inside collection JSON files to public GitHub repositories.
6. **Confusing 401 with 403**: `401 Unauthorized` means *unauthenticated* (who are you?); `403 Forbidden` means *unauthorized* (you are authenticated, but lack permission).
7. **Assuming test execution order without chaining**: Depending on alphabetical folder ordering instead of explicit test chaining or collection runner sequences.

---

22. Interview questions — 15 essentials

Q1. What is Postman used for in backend engineering?

Answer: Postman is an API platform used for constructing HTTP requests, testing RESTful/GraphQL endpoints, inspecting response status and payloads, automating regression suites via JavaScript assertions, managing environments, and integrating API checks into CI/CD pipelines via Newman.

Q2. What is a Postman collection?

Answer: A structured, shareable JSON-exportable container that groups related API requests, organized into hierarchical folders, with shared variables, authentication configurations, and pre/post-request test scripts.

Q3. What is the difference between an Environment and Global variables?

Answer: Environment variables are scoped to a specific target deployment context (e.g. `local`, `staging`, `prod`), whereas Global variables are accessible across all collections and environments within a Postman workspace.

Q4. What is the difference between path parameters and query parameters?

Answer: Path parameters identify a specific resource in the hierarchical URI path (e.g. `/api/orders/:id`). Query parameters appear after `?` and filter, paginate, sort, or modify the response representation (e.g. `?page=0&size=10&status=PAID`).

Q5. What is the difference between HTTP status codes 401 and 403?

Answer: `401 Unauthorized` indicates that authentication is required and has failed or not been provided (unauthenticated identity). `403 Forbidden` indicates that the server understands the caller's verified identity, but refuses to authorize access because the user lacks required roles or permissions.

Q6. How do you automate response assertions in Postman?

Answer: In the **Scripts** → **Post-response** tab of a request, write JavaScript tests using `pm.test()` and Chai-based assertions via `pm.expect()`, validating status codes, headers, response times, and JSON body properties.

Q7. How do you pass data between chained requests in a collection?

Answer: In the post-response script of the first request, extract the value from the JSON payload (`pm.response.json().id`) and persist it into a collection variable using `pm.collectionVariables.set("key", value)`. In the subsequent request, reference it using `{{key}}`.

Q8. How do you test a protected endpoint requiring JWT Bearer authentication?

Answer: Configure the Authorization tab with type **Bearer Token**, set the token value to an environment variable (`{{accessToken}}`), and execute test cases for: (1) no token (401), (2) expired/tampered token (401), (3) valid token with insufficient role (403), and (4) valid token with correct role (200/201).

Q9. How do you test backend validation logic in Postman?

Answer: Construct negative test cases submitting missing required fields, negative values, type mismatches, and malformed JSON, asserting that the API responds with `400 Bad Request` or `422 Unprocessable Entity` alongside a standardized problem details error payload.

Q10. How do you test paginated APIs in Postman?

Answer: Send requests with varying `page` and `size` parameters, asserting that: (1) the array length matches the requested page size, (2) page metadata (`totalElements`, `totalPages`) is mathematically consistent, and (3) boundary conditions (e.g. `page=-1` or `size=10000`) are handled gracefully without crashing.

Q11. What is a pre-request script and when would you use it?

Answer: A JavaScript block executed before the HTTP request is transmitted over the wire. Common uses include generating dynamic timestamps, computing cryptographic request signatures (HMAC), creating random UUIDs, or refreshing an expired auth token.

Q12. What is a post-response script?

Answer: A JavaScript block executed immediately after the response arrives. Used to assert contract requirements, extract tokens or generated resource IDs, and set variables for downstream requests.

Q13. How do you run Postman tests inside CI/CD pipelines?

Answer: Export the collection and environment as JSON files and run them headless using **Newman** (e.g. `newman run collection.json -e env.json --bail`). If any assertion fails, Newman exits with a nonzero status code, failing the CI pipeline step.

Q14. Is Postman suitable for load and stress testing?

Answer: No. Postman is designed for functional, regression, and contract testing. It lacks distributed virtual user simulation, realistic arrival rate control, and statistical latency percentile modeling ($p95$, $p99$). Tools like k6, JMeter, or Gatling should be used for load testing.

Q15. How would you investigate an intermittent 500 error using Postman?

Answer: Open the Postman Console to inspect full outgoing wire headers and incoming server headers. Extract the `X-Trace-Id` or correlation ID from the response headers, correlate it with backend server log traces, and verify whether the issue stems from database pool exhaustion, external API timeouts, or unhandled exceptions.

---

23. Hands-on capstone

Create an **Orders API** collection implementing an end-to-end automated test suite:

1. **Check API Health**: `GET {{baseUrl}}/actuator/health` → Assert `200 OK` and `status == "UP"`.
2. **Create Order**: `POST {{baseUrl}}/api/orders` → Assert `201 Created`, extract `body.id`, save to `{{orderId}}`.
3. **Get Order by ID**: `GET {{baseUrl}}/api/orders/{{orderId}}` → Assert `200 OK`, verify `id == {{orderId}}`.
4. **List Orders with Pagination**: `GET {{baseUrl}}/api/orders?page=0&size=10` → Assert `200 OK`, verify array structure.
5. **Submit Invalid Order**: `POST {{baseUrl}}/api/orders` with negative quantity → Assert `400 Bad Request`.
6. **Access Endpoint without Auth**: `GET {{baseUrl}}/api/orders` without headers → Assert `401 Unauthorized`.
7. **Access Forbidden Resource**: `GET {{baseUrl}}/api/admin/metrics` with user token → Assert `403 Forbidden`.

8. Delete Order: `DELETE {{baseUrl}}/api/orders/{{orderId}}` → Assert `204 No Content` or `200 OK`.

Final Mastery Checklist

- [ ] Construct requests with the correct HTTP method, URL, headers and body.
- [ ] Explain standard HTTP status codes (`200`, `201`, `204`, `400`, `401`, `403`, `404`, `409`, `429`, `500`).
- [ ] Organize requests into structured collections and folders.
- [ ] Configure environments and scope variables properly.
- [ ] Write robust assertions using `pm.test` and `pm.expect`.
- [ ] Chain requests dynamically using collection variables.
- [ ] Test authentication (`401`) and authorization (`403`) scenarios.
- [ ] Cover invalid input, boundary conditions, and schema validation.
- [ ] Execute collections headlessly using Newman in CI/CD.
- [ ] Explain the architectural differences between functional API testing and load testing.

Completion Protocol

1. Authoritative source file: `content/postman/01-postman-api-testing-workflows-production.md`.
2. All three layers present: Fundamentals & Mental Models, Practical Engineering & Workflows, and Production Engineering & Interview Mastery.
3. Single final Postman handbook PDF deliverable: `docs/postman/BEH_POSTMAN_COMPLETE_HANDBOOK.pdf`.
4. Update `README.md` progress tracker.
5. Commit using exact message: `Postman done`.